

Лабораторная работа № 2. Структуры данных.

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

Информация

- Абакумова Олеся Максимовна
- Студентка
- Российский университет дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



Цель работы

Основная цель работы — изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

Выполнение лабораторной работы

Кортеж (Tuple) — структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы). Синтаксис определения кортежа: `(element1, element2, ...)`

Примеры кортежей

```
[6]: # пустой кортеж:
[6]: ()

[7]: # кортеж из элементов типа String:
[7]: favoritelang = ("Python", "Julia", "R")
[7]: ("Python", "Julia", "R")

[8]: # кортеж из целых чисел:
[8]: x1 = (1, 2, 3)
[8]: (1, 2, 3)

[9]: # кортеж из элементов разных типов:
[9]: x2 = (1, 2.0, "tep")
[9]: (1, 2.0, "tep")

[10]: # именованный кортеж:
[10]: x3 = (a=2, b=1+2)
[10]: (a = 2, b = 3)
```

Рис. 1: Примеры кортежей

Примеры операций над кортежами

```
[11]: # длина кортежа x2:
      length(x2)
[11]: 3

[12]: # обратиться к элементам кортежа x2:
      x2[1], x2[2], x2[3]
[12]: (1, 2.0, "tmp")

[13]: # произвести какую-либо операцию (сложение)
      # с вторым и третьим элементами кортежа x1:
      c = x1[2] + x1[3]
[13]: 5

[14]: # обращение к элементам именованного кортежа x3:
      x3.a, x3.b, x3[2]
[14]: (2, 3, 3)

[15]: # проверка вхождения элементов tmp и 0 в кортеж x2
      # (два способа обращения к методу in()):
      in("tmp", x2), 0 in x2
[15]: (True, False)
```

Рис. 2: Операции над кортежами

Словарь — неупорядоченный набор связанных между собой по ключу данных. Синтаксис определения словаря: `Dict(key1 => value1, key2 => value2, ...)`


```
Словари

[14]: # создать словарь с именем phonebook:
phonebook = Dict{String, Any} => {"067-5389", "333-5544"}, "Бугагаторы" => "555-2368"}

[15]: Dict{String, Any} with 2 entries:
"Бугагаторы" => "555-2368"
"Ваняа Р.Д." => {"067-5389", "333-5544"}

[17]: # вывести ключи словаря:
keys(phonebook)

[17]: KeyList for a Dict{String, Any} with 2 entries. Keys:
"Бугагаторы"
"Ваняа Р.Д."

[18]: # вывести значения элементов словаря:
values(phonebook)

[19]: ValueIterator for a Dict{String, Any} with 2 entries. Values:
"555-2368"
{"067-5389", "333-5544"}

[19]: # вывести пары элементов в словарь под названием "пары - значения":
pairs(phonebook)

[19]: Dict{String, Any} with 2 entries:
"Бугагаторы" => "555-2368"
"Ваняа Р.Д." => {"067-5389", "333-5544"}

[20]: # проверить наличие ключа в словаре:
haskey(phonebook, "Ваняа Р.Д.")

[20]: true

[21]: # получить элемент в словаре:
phonebook["Сашаа П.С."] => "555-2344"

[21]: "555-3544"

[22]: # вывести ключ и значение с тем значением из словаря
pop!(phonebook, "Ваняа Р.Д.")

[22]: {"067-5389", "333-5544"}

[24]: # Объединение словарей (функция merge!!!)
a = Dict{String, Real} => 0.8, "bar" => 42.8;
b = Dict{String, Real} => 17, "bar" => 13.6;
merge(a, b, merge{b, a})

[24]: Dict{String, Real}{"bar" => 13.6, "bar" => 17, "foo" => 0.8}, Dict{String, Real}{"bar" => 42.8, "bar" => 17, "foo" => 0.8})
```

Рис. 3: Примеры словарей и операций над ними

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа. Возможные операции над множествами: объединение, пересечение, разность; принадлежность элемента множеству.

Синтаксис определения множества: `Set([itr])` где `itr` — набор значений, сгенерированных данным итерируемым объектом или пустое множество.

Множества

```
[25]: # создать множество из четырёх целочисленных значений:  
A = Set([1, 3, 4, 5])
```

```
[25]: Set{Int64} with 4 elements:  
5  
4  
3  
1
```

```
[26]: # создать множество из 11 символьных значений:  
B = Set("abrakadabra")
```

```
[26]: Set{Char} with 5 elements:  
'a'  
'd'  
'r'  
'k'  
'b'
```

```
[27]: # проверка эквивалентности двух множеств:  
S1 = Set([1,2]);  
S2 = Set([3,4]);  
issetequal(S1,S2)
```

```
[27]: false
```

```
[28]: S3 = Set([1,2,2,3,1,2,3,2,1]);  
S4 = Set([2,3,1]);  
issetequal(S3,S4)
```

```
[28]: true
```

```
[29]: # объединение множеств:  
C = union(S1,S2)
```

```
[29]: Set{Int64} with 4 elements:  
4  
2  
3  
1
```

```
[30]: # пересечение множеств:  
D = intersect(S1,S3)
```

```
[30]: Set{Int64} with 2 elements:  
2  
1
```

Рис. 4: Примеры множеств и операций над ними

```
[31]: # разность множеств:  
E = setdiff(S3,S1)  
[31]: Set{Int64} with 1 element:  
3  
[32]: # проверка вхождения элементов одного множества в другое:  
issubset(S1,S4)  
[32]: true  
[33]: # добавление элемента в множество:  
push!(S4, 99)  
[33]: Set{Int64} with 4 elements:  
2  
99  
3  
1  
[34]: # удаление последнего элемента множества:  
pop!(S4)  
[34]: 2
```

Рис. 5: Примеры множеств и операций над ними

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов. Общий синтаксис одномерных массивов:

```
array_name_1 = [element1, element2, ...]
```

```
array_name_2 = [element1 element2 ...]
```

Массивы

```
[35]: # создание пустого массива с абстрактным типом:
empty_array_1 = []

[35]: Any[]

[37]: # создание пустого массива с конкретным типом:
empty_array_2 = [Int64]()

[37]: Int64[]

[38]: empty_array_3 = {Float64}()

[38]: Float64[]

[39]: # вектор-столбец:
a = [1, 2, 3]

[39]: 3-element Vector{Int64}:
 1
 2
 3

[40]: # вектор-строка:
b = [1 2 3]

[40]: 1×3 Matrix{Int64}:
 1 2 3

[41]: # многомерные массивы (матрицы):
A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]

[41]: 3×3 Matrix{Int64}:
 1 4 7
 2 5 8
 3 6 9

[42]: B = [[1 2 3]: [4 5 6]: [7 8 9]]

[42]: 3×3 Matrix{Int64}:
 1 2 3
 4 5 6
 7 8 9

[45]: # одномерный массив из 8 элементов (массив 51 \times 85)
# со значениями, случайно распределёнными на интервале [0, 1]:
c = rand(1,8)

[45]: 1×8 Matrix{Float64}:
 0.555361 0.188253 0.902527 0.818364 - 0.981847 0.118177 0.111858
```

Рис. 6: Примеры массивов

```
[49]: # многомерный массив $2 (times 35 (2 строки, 3 столбца) элементов
# со значениями, случайно распределёнными на интервале [0, 1]:
C = rand(2,3)

[49]: 2x3 Matrix{Float64}:
 0.908834  0.852383  0.279751
 0.0869465 0.122167  0.542962

[50]: # трёхмерный массив:
D = rand(4, 3, 2)

[50]: 4x3x2 Array{Float64, 3}:
[:, :, 1] =
 0.349258  0.20746  0.209247
 0.878426  0.92443  0.793306
 0.987893  0.088688 0.527845
 0.596582  0.582863 0.393346

[:, :, 2] =
 0.309682  0.961072  0.639578
 0.855543  0.72326  0.803434
 0.799612  0.0782893 0.261948
 0.0732649 0.488365 0.0923995

[51]: # массив из квадратных корней всех целых чисел от 1 до 10:
roots = [sqrt(i) for i in 1:10]

[51]: 10-element Vector{Float64}:
 1.0
 1.4142135623730951
 1.7320508075688772
 2.0
 2.23606797749979
 2.449489742783178
 2.6457513110645907
 2.8284271247461903
 3.0
 3.1622776601683795

[52]: # массив с элементами вида 3^x*2,
# где x - нечётное число от 1 до 9 (включительно)
ar_1 = {3^i*2 for i in 1:2:9}

[52]: 5-element Vector{Int64}:
 3
 27
 75
 147
 243

[53]: # массив квадратов элементов, если квадрат не делится на 5 или 4:
ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

[53]: 4-element Vector{Int64}:
 1
 9
 49
 81
```

Некоторые операции для работы с массивами:

- `length(A)` — число элементов массива `A`;
- `ndims(A)` — число размерностей массива `A`;
- `size(A)` — кортеж размерностей массива `A`;

- `size(A, n)` — размерность массива `A` в заданном направлении;
- `copy(A)` — создание копии массива `A`;
- `ones()`, `zeros()` — создать массив с единицами или нулями соответственно;
- `fill(value,array_name)` — заполнение массива заранее определенным значением;

- `sort()` — сортировка элементов;
- `collect()` — вернуть массив всех элементов в коллекции или итераторе;
- `reshape()` — изменение размера массива;
- `transpose()` — транспонирование массива;

Некоторые операции для работы с массивами

```
[55]: # одномерный массив из пяти единиц:
      ones(5)

[55]: 5-element Vector{Float64}:
      1.0
      1.0
      1.0
      1.0
      1.0

[56]: # двумерный массив 2x3 из единиц:
      ones(2,3)

[56]: 2x3 Matrix{Float64}:
      1.0  1.0  1.0
      1.0  1.0  1.0

[57]: # одномерный массив из 4 нулей:
      zeros(4)

[57]: 4-element Vector{Float64}:
      0.0
      0.0
      0.0
      0.0

[58]: # заполнить массив 3x2 цифрами 3.5
      fill(3.5,(3,2))

[58]: 3x2 Matrix{Float64}:
      3.5  3.5
      3.5  3.5
      3.5  3.5

[59]: # заполнение массива посредством функции repeat():
      repeat([1,2],3,3)
      repeat([1 2],3,3)

[59]: 3x6 Matrix{Int64}:
      1  2  1  2  1  2
      1  2  1  2  1  2
      1  2  1  2  1  2

[60]: # преобразование одномерного массива из целых чисел от 1 до 12
      # a - двумерный массив 2x6
      a = collect(1:12)
      b = reshape(a,(2,6))

[60]: 2x6 Matrix{Int64}:
      1  3  5  7  9  11
      2  4  6  8  10 12

[61]: # транспонирование
      b

[61]: 2x6 Matrix{Int64}:
      1  3  5  7  9  11
      2  4  6  8  10 12
```

```
[62]: # транспонирование
      c = transpose(b)

[62]: 6x2 transpose{::Matrix{Int64}} with eltype Int64:
       1  2
       3  4
       5  6
       7  8
       9 10
      11 12

[63]: # массив 10x5 целых чисел в диапазоне [10, 20]:
      ar = rand(10:20, 10, 5)

[63]: 10x5 Matrix{Int64}:
      11 17 19 14 20
      12 16 16 10 11
      18 18 19 18 13
      12 13 11 12 19
      18 11 10 10 10
      10 11 16 10 19
      20 13 18 15 19
      18 13 20 20 20
      17 17 16 19 19
      13 18 19 19 19

[64]: # выбор всех значений строки в столбце 2:
      ar[:, 2]

[64]: 10-element Vector{Int64}:
      17
      16
      18
      13
      11
      11
      13
      13
      17
      18

[65]: # выбор всех значений в столбцах 2 и 5:
      ar[:, [2, 5]]

[65]: 10x2 Matrix{Int64}:
      17 20
      16 11
      18 13
      13 19
      11 10
      11 19
      13 19
      13 20
      17 19
      18 19
```

Рис. 9: Примеры

```
[66]: # все значения строк в столбцах 2, 3 и 4:
      ar[:, 2:4]

[66]: 10x3 Matrix{Int64}:
      17 19 14
      16 16 10
      18 19 18
      13 11 12
      11 10 10
      11 16 10
      13 10 15
      13 20 20
      17 16 19
      18 19 19

[67]: # значения в строках 2, 4, 6 и в столбцах 1 и 5:
      ar[[2, 4, 6], [1, 5]]

[67]: 3x2 Matrix{Int64}:
      12 11
      12 19
      10 19

[68]: # значения в строке 1 от столбца 3 до последнего столбца:
      ar[1, 3:end]

[68]: 3-element Vector{Int64}:
      19
      14
      20

[70]: # сортировка по столбцам:
      sort(ar, dims=1)

[70]: 10x5 Matrix{Int64}:
      10 11 10 10 10
      11 11 10 10 11
      12 13 11 10 13
      12 13 16 12 19
      13 13 16 14 19
      17 16 16 15 19
      18 17 19 18 19
      18 17 19 19 19
      18 18 19 19 20
      20 18 20 20 20

[71]: # сортировка по строкам:
      sort(ar, dims=2)

[71]: 10x5 Matrix{Int64}:
      11 14 17 19 20
      10 11 12 16 16
      13 18 10 18 19
      11 12 12 13 19
      10 10 10 11 18
      10 10 11 16 19
      10 13 15 19 20
      13 18 20 20 20
      16 17 17 19 19
      13 18 19 19 19
```

Рис. 10: Примеры

```
[72]: # поэлементное сравнение с числом
# (результат - массив логических значений):
ar -> 14

[72]: 10x5 BitMatrix:
0 1 1 0 1
0 1 1 0 0
1 1 1 1 0
0 0 0 0 1
1 0 0 0 0
0 0 1 0 1
1 0 0 1 1
1 0 1 1 1
1 1 1 1 1
0 1 1 1 1

[73]: # возврат индексов элементов массива, удовлетворяющих условию:
findall(ar -> 14)

[73]: 29-element Vector{CartesianIndex{2}}:
 CartesianIndex(3, 1)
 CartesianIndex(5, 1)
 CartesianIndex(7, 1)
 CartesianIndex(8, 1)
 CartesianIndex(9, 1)
 CartesianIndex(1, 2)
 CartesianIndex(2, 2)
 CartesianIndex(3, 2)
 CartesianIndex(9, 2)
 CartesianIndex(10, 2)
 CartesianIndex(1, 3)
 CartesianIndex(2, 3)
 CartesianIndex(3, 3)
 ⋮
 CartesianIndex(3, 4)
 CartesianIndex(7, 4)
 CartesianIndex(8, 4)
 CartesianIndex(9, 4)
 CartesianIndex(10, 4)
 CartesianIndex(1, 5)
 CartesianIndex(4, 5)
 CartesianIndex(6, 5)
 CartesianIndex(7, 5)
 CartesianIndex(8, 5)
 CartesianIndex(9, 5)
 CartesianIndex(10, 5)
```

Рис. 11: Примеры

Задания для самостоятельной работы

1. Даны множества: $A = 0, 3, 4, 9$, $B = 1, 3, 4, 7$, $C = 0, 1, 2, 4, 7, 8, 9$.
Найдем $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$.
2. Приведем свои примеры с выполнением операций над множествами элементов разных типов.

Задания для самостоятельной работы

```
Задание для самостоятельного выполнения

[74]: # 1. Операции над множествами
      A = Set{0, 3, 4, 9}
      B = Set{1, 3, 4, 7}
      C = Set{0, 1, 2, 4, 7, 8, 9}

[74]: Set{Int64} with 7 elements:
      8
      4
      7
      2
      9
      8
      1

[78]: D = union(intersect(A, B), intersect(A, C), intersect(A, C), intersect(B, C))

[78]: Set{Int64} with 6 elements:
      8
      4
      7
      9
      3
      1

[79]: # 2. Примеры операций над множествами разных типов
      # Множество строк
      set1 = Set{("apple", "banana", "orange")}

[79]: Set{String} with 3 elements:
      "orange"
      "banana"
      "apple"

[80]: # Множество чисел
      set2 = Set{1, 2, 3, 4, 5}

[80]: Set{Int64} with 5 elements:
      5
      4
      2
      3
      1

[81]: # Объединение множеств разных типов (нельзя смешивать типы в Julia)
      # Поэтому создадим множество Any
      set_mixed = Set{Any}{"text", 42, 3.14, :symbol}

[81]: Set{Any} with 4 elements:
      :symbol
      3.14
      42
      "text"

[82]: println("Смешанное множество: ", set_mixed)

      Смешанное множество: Set{Any{::Symbol, 3.14, 42, "text"}}
```

Рис. 12: Задание 1-2. Работа с множествами и свои примеры работы с ними

3. Создадим разными способами массивы и векторы.

Задания для самостоятельной работы

[illegible]

Рис. 13: Пункт 3.1-3.9

Задания для самостоятельной работы

```
[95]: using Statistics

[100]: # 3.10 Вектор  $y = e^{ix} \cdot \cos(x)$  для  $x = 3, 3.1, 3.2, \dots, 6$ 
      x_range = 3:0.1:6 # Диапазон от 3 до 6 с шагом 0.1
      y = exp.(x_range) .* cos.(x_range) # Поэлементная операция

[100]: 31-element Vector{Float64}:
      -19.88453884146807
      -22.178753389342127
      -24.49696732801293
      -26.77318244299338
      -28.969237768993574
      -31.01186439374516
      -32.818774768338504
      -34.383368118377369
      -35.357193618538025
      -35.8626373238767
      -35.68773248811913
      -34.68564225368887
      -32.693695428321746
      25.048704998273804
      42.09926106253839
      61.99863827866454
      84.92966736258268
      111.0615866426258
      140.5256758527875
      173.48577648857734
      209.7349424783487
      249.46844853855468
      292.4887687371223
      338.5643778585117
      387.36834829993876

[100]: mean_y = mean(y) # Среднее значение
      println("3.10 Среднее значение y: ", round(mean_y, digits=4))
      3.10 Среднее значение y: 53.1137

[101]: # 3.11 Вектор  $(x^i, y^j)$  для заданных параметров
      x_vals = 0.1
      y_vals = 0.2
      i_range = 3:3:36 #  $i = 3, 6, 9, \dots, 36$ 

[101]: 3:3:36

[102]: j_range = 1:3:34 #  $j = 1, 4, 7, \dots, 34$ 

[102]: 1:3:34

[103]: # Создаем массивы  $x^i$  и  $y^j$ 
      x_powers = [x^i for x in x_vals, i in i_range]

[103]: 12-element Vector{Float64}:
      0.010000000000000002
      1.0000000000000004e-6
      1.0000000000000005e-9
      1.0000000000000006e-12
      1.0000000000000006e-15
      1.0000000000000006e-18
      1.0000000000000012e-21
      1.0000000000000012e-24
      1.0000000000000015e-27
      1.0000000000000017e-30
      1.0000000000000019e-33
      1.000000000000002e-36
```

Задания для самостоятельной работы

```
[104]: y.powers = [y**j for j in y.xvals, 1 in 1, range(1)]

[104]: 12-element Vector{Float64}:
 8.2
 8.800000000000000e-05
 1.200000000000000e-5
 1.420000000000000e-7
 8.700000000000000e-10
 6.550000000000000e-12
 3.240000000000000e-14
 1.040000000000000e-16
 3.250000000000000e-18
 2.000000000000000e-20
 2.340000000000000e-22
 1.170000000000000e-24

[105]: arr11 = vcat(y.powers[1], y.powers[1])
println("3.11 Вектор: ", length(arr11), ", уникальные значения: ", unique(arr11))

3.11 Вектор: 24, уникальные значения: 18, 0.000000000000000e+00, 1.000000000000000e-04, 1.000000000000000e-06, 1.000000000000000e-12,
1.000000000000000e-20, 1.000000000000000e-18, 1.000000000000000e-24, 1.000000000000000e-26, 1.000000000000000e-27, 1.000000000000000e-29,
1.000000000000000e-30, 1.000000000000000e-32, 1.000000000000000e-34, 8.2, 8.000000000000000e-05, 1.200000000000000e-05, 1.420000000000000e-07, 8.700000000000000e-10,
6.550000000000000e-12, 3.240000000000000e-14, 1.040000000000000e-16, 3.250000000000000e-18, 2.000000000000000e-20, 2.340000000000000e-22, 1.170000000000000e-24

[106]: # 3.12 Вектор x^j для j = 1, 2, ..., M, M=25
arr12 = [x^j for j in 1:M, x in 1:25]
println(arr12)

12.0, 2.0, 2.000000000000000e+00, 4.0, 0.0, 10.000000000000000e+00, 10.20714205714205, 22.0, 50.000000000000000e+00, 100.0, 100.0000000000000e+00, 241.39
302.131113, 630.153661518050, 1176.207142057142, 2186.531122252129, 3956.8, 7719.1176190476, 12561.555555555555, 27506.105263157890,
54226.8, 99904.10091288897, 190608.10310310302, 364732.0009161217, 699950.0000000000, 1.341177360e+06

[107]: # 3.13 Вектор ("x1", "x2", ..., "xM"), M=10
M_rows = 10
arr13 = ["x1" for i in 1:M, row in 1:M_rows] # 10x10 матрица строк

[107]: 10-element Vector{String}:
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
 "x1"
```

Рис. 15: Пункт 3.11-3.13

Задания для самостоятельной работы

```
1180: using Random
1181: using Primes

1182: # 3.14) Создать случайные векторы
Random.seed!(123) # Для воспроизводимости результатов
N = 200
x = rand{0:999, N}; # Вектор x из случайных целых от 0 до 999

1183: y = rand{0:999, N}; # Вектор y из случайных целых от 0 до 999

1184: # Вектор z
vec1 = [y[i+1] - x[i] for i in 1:N-1]

1185: 240-element Vector{Int64}:
-311
-124
-794
460
228
521
426
-11
187
208
290
-562
887
-188
778
305
286
-398
-79
818
-132
-394
978
357
531

1186: # Вектор z
vec2 = [x[i+1] + 2*y[i+1] - x[i+1] for i in 1:N-1]

1187: 240-element Vector{Int64}:
889
2178
140
870
1261
479
1350
1366
708
942
2154
451
608
490
722
```

Рис. 16: Пункт 3.14

Задания для самостоятельной работы

```
[132]: # Backup 2
vec3 = [sinly[i]] / cos(x[i+1]) for i in 1:n-1]

[132]: 240-element Vector{Float64}:
 0.571958432642845
-1.4219588205712436
-2.787213847838385
-1.0486185578043187
 0.33723818254481256
-0.831871621143289
 0.861817879696797966
 2.838478404548142
-1.83742895647486
-0.6202102056162984
 1.850184212140219
-1.178474388677485
 0.452427277746794
-1.8119887724553
 2.852418166661387
-1.9048223874189785
-1.12684438251166386
-0.9807237302792472
-0.4288421815612119
-0.8489412571423979
-0.7818361216795619
-2.146571596187484
-0.365792750792751
-0.414495952366843
 0.88879814793338226

[134]: # sum
sum_val = sum(exp(-x[i+1]) / (x[i] + 10) for i in 1:n)

[134]: 0.0001785210565887641

[136]: # Элементы массива y > 600
y_greater_600 = y[y .> 600]

[136]: 114-element Vector{Int64}:
 650
 753
 911
 678
 822
 787
 655
 824
 847
 805
 823
 788
 679
 1
 978
 999
 982
 945
 888
 696
 695
 638
 736
 766
 842
 922

[137]: println("5. Элементы y > 600: ", length(y_greater_600), " элементов")
5. Элементы y > 600: 114 элементов

+138. println("Первые 10: ", y_greater_600[1:min(10, end)])
Первые 10: [650, 753, 911, 678, 822, 787, 655, 824, 847, 805]
```

Рис. 17: Пункт 3.14

Задания для самостоятельной работы

```
[139]: # Идентифицируем y > 600
indices_y_600 = findall(y > 600)

[139]: 114-element Vector{Int64}:
       5
       6
       7
       8
       9
      10
      11
      12
      14
      17
      21
      22
      23
      27
      28
      29
      32
      33
      35
      36
      38
      40
      45
      47
      49
      50
      51
      52
      53
      54
      55
      56
      57
      58
      59
      60
      61
      62
      63
      64
      65
      66
      67
      68
      69
      70
      71
      72
      73
      74
      75
      76
      77
      78
      79
      80
      81
      82
      83
      84
      85
      86
      87
      88
      89
      90
      91
      92
      93
      94
      95
      96
      97
      98
      99
      100
      101
      102
      103
      104
      105
      106
      107
      108
      109
      110
      111
      112
      113
      114

[141]: # Находим x, соответствующие y > 600
x_corresponding = x[y > 600]

[141]: 114-element Vector{Int64}:
      525
      390
      44
      933
      980
      327
      526
      836
      405
      720
      276
      983
      890
      1
      95
      858
      984
      834
      463
      321
      136
      280
      222
      2
      391
      885

[143]: # Вектор d
x_mean = mean(x)
w0d = sqrt(abs.(x - x_mean))

[143]: 250-element Vector{Float64}:
 5.741428293761244
 9.06976495931283
 20.048693877268956
 17.263729182214652
 6.070826262575385
 9.061313844238123
 21.072161730588636
```

Рис. 18: Пункт 3.14

Задания для самостоятельной работы

```
[145]: # Элементы y, отстоящие от максимума не более чем на 200
      y_max = maximum(y)

[146]: 999

[148]: y_close_to_max = y[abs.(y .- y_max) .<= 200]

[148]: 57-element Vector{Int64}:
      911
      922
      924
      927
      995
      923
      952
      917
      961
      920
      907
      911
      990
      1
      894
      853
      872
      884
      978
      999
      902
      945
      888
      926
      882
      922

[147]: println("9. Элементы y, отстоящие от максимума < 200: ", length(y_close_to_max))
      9. Элементы y, отстоящие от максимума < 200: 57

[149]: # Чётные и нечётные элементы x
      even_count = count(iseven, x)

[149]: 129

[150]: odd_count = count(isodd, x)

[150]: 121

[151]: println("10. Чётные элементы x: ", even_count)
      10. Чётные элементы x: 129

[152]: println("10. Нечётные элементы x: ", odd_count)
      Нечётные элементы x: 121

[153]: # Элементы x, кратные 7
      multiples_of_7 = count(x -> x % 7 == 0, x)

[153]: 39

[154]: println("11. Элементы x, кратные 7: ", multiples_of_7)
      11. Элементы x, кратные 7: 39

[155]: # Сортировка x в порядке возрастания y
      x_sorted_by_y = x[sortperm(y)]

[155]: 250-element Vector{Int64}:
      471
      799
      152
      34
      100
```

Рис. 19: Пункт 3.14

4. Создадим массив `squares`, в котором будут храниться квадраты всех целых чисел от 1 до 100.

Задания для самостоятельной работы

```
[157]: # Top-10 наибольшие элементы x
x_top10 = sort(x, reverse)[1:min(10, end)]

[157]: 10-element Vector{Int64}:
 996
 993
 993
 986
 983
 974
 959
 953
 949
 943

[159]: # Уникальные элементы x
x_unique = unique(x)

[159]: 227-element Vector{Int64}:
 521
 586
 699
 190
 525
 590
 44
 933
 589
 327
 536
 836
 89
 1
 136
 152
 209
 492
 371
 126
 222
 679
 2
 75
 291
 893

[160]: println("14. Уникальные элементы x: ", length(x_unique), " элементов")
14. Уникальные элементы x: 227 элементов

[161]: # 4. Массив квадратов чисел от 1 до 100
squares = [i^2 for i in 1:100]

[161]: 100-element Vector{Int64}:
 1
 4
 9
 16
 25
 36
 49
 64
 81
 100
 121
 144
 169
 1
 169
```

Рис. 20: Конец пункта 3.14 и задание 4

Задания для самостоятельной работы

5. Подключим пакет `Primes` (функции для вычисления простых чисел). Сгенерируем массив `myprimes`, в котором будут храниться первые 168 простых чисел. Определим 89-е наименьшее простое число. Получим срез массива с 89-го до 99-го элемента включительно, содержащий наименьшие простые числа.
6. Вычислим выражения.

Задания для самостоятельной работы

```
[183]: # 3. Простые числа с использованием пакета Primes
myprimes = primes(1000)[1:361] # Первые 360 простых чисел

[183]: 168-element Vector{Int64}:
       2
       3
       5
       7
      11
      13
      17
      19
      23
      29
      31
      37
      41
       :
     910
     929
     937
     941
     947
     953
     967
     971
     977
     983
     991
     997

*165. println("89-е наименьшее простое число: ", myprimes[89])
      89-е наименьшее простое число: 461

*166. println("Средн с 89-го до 99-го: ", myprimes[89:99])
      Средн с 89-го до 99-го: 1461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523

*167. # 6. Вычисление сумм
# 6.1) i=1 до 100
sum1 = sum(i^2 + 4*i^2 for i in 1:100)

[187]: 24855900

[189]: # 6.2) i=1 до M=25
M = 25
sum2 = sum(2^i/i + 3^i/i^2 for i in 1:M)

[189]: 2.1291704308143082e0

[190]: # 6.3) Сложная сумма
sum6 = 1
num = 1

[190]: 1

[191]: for i in 2:2:38
      num *= i/(i+1)
      sum6 += num
    end

[192]: print(sum6)
      6.976346137897618

[1:]
```

Рис. 21: Задание 5-6

Выводы

В процессе выполнения данной лабораторной работы я изучила несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.